

Relatório Técnico — Vitrine Virtual da Feira do Largo da Ordem

Vitrine Virtual — Feira do Largo da Ordem

Projeto da disciplina **Mobile Development iOS — PUCPR (2026)** — Avaliação Somativa SwiftUI. App iOS nativo em **SwiftUI** que simula uma vitrine virtual dos produtos artesanais vendidos na tradicional feira de domingo do Largo da Ordem, em Curitiba. O foco principal do projeto é a implementação rigorosa das diretrizes de **Acessibilidade (A11y)** da Apple.

Links da entrega

- **Vídeo de apresentação (YouTube — Não Listado):** *(adicionar link após gravação)*
- **Repositório do projeto:**
https://github.com/LucasStop/ios_development/tree/main/Somativa3/VitrineLargoOrdem (https://github.com/LucasStop/ios_development/tree/main/Somativa3/VitrineLargoOrdem)

Componentes do grupo

- Lucas Stopinski da Silva
- Lucas Bruno e Silva

Como rodar

1. Abrir `VitrineLargoOrdem.xcodeproj` no Xcode 15+ (testado com Xcode 26).
2. Selecionar um simulador iPhone ou iPad (iOS 17+).
3. `Cmd+R` para executar.

O projeto é gerado a partir de `project.yml` via `xcodegen` (<https://github.com/yonaskolb/XcodeGen>). Para regenerar: `xcodegen generate` na raiz do projeto.

Arquitetura

MVVM (Model-View-ViewModel) — padrão idiomático do SwiftUI:

VitrineLargoOrdem/	
└ VitrineLargoOrdemApp.swift	– Ponto de entrada (@main)
└ Models/	
└ ProdutoArtesanal.swift	– struct Identifiable com preço formatado/acess
└ Data/	
└ ProdutosMockData.swift	– Catálogo estático com 12 produtos típicos
└ ViewModels/	
└ VitrineViewModel.swift	– ObservableObject (lista, busca, favoritos)
└ Views/	
└ VitrineView.swift	– Tela principal (LazyVGrid + .searchable)
└ ProdutoCardView.swift	– Card componentizado
└ BotaoFavoritoView.swift	– Botão extraído (44x44 mínimo)
└ DetalhesProdutoView.swift	– Tela de detalhes com sortPriority

- **Models** não conhece UIKit/SwiftUI — pura lógica de domínio.
- **ViewModel** anotado com `@MainActor` para evitar concorrência indevida; expõe `produtosFiltrados` derivado de `produtos + termoBusca`.
- **Views** consomem o ViewModel via `@StateObject` e propagam ações via closures (`aoFavoritar`).

Acessibilidade (A11y) — destaque do projeto

Requisito	Implementação
VoiceOver — imagens	<code>.accessibilityLabel("Imagem ilustrativa de \(nome), produzida por \(artesão)")</code> em todas as imagens dos produtos
VoiceOver — preços	Propriedade <code>precoAcessivel</code> no model retorna <code>"Preço: 45 reais"</code> em vez de <code>"R cifrão 45 ponto 00"</code>
Touch targets	<code>BotaoFavoritoView</code> força <code>.frame(minWidth: 44, minHeight: 44)</code> + <code>.contentShape(Rectangle())</code> para área clicável completa
Dynamic Type	Toda tipografia usa fontes semânticas (<code>.headline</code> , <code>.body</code> , <code>.subheadline</code>); <code>@ScaledMetric</code> no tamanho das imagens dos cards; nenhum <code>.frame(height:)</code> fixo em textos
Ordem de leitura nos detalhes	<code>.accessibilitySortPriority</code> ordena: Nome (10) → Metadados (8) → Descrição (6) → Imagem (5) → Botão de contato (4)
Estado de favorito	Label dinâmico: "Adicionar X aos favoritos" / "Remover X dos favoritos", com <code>.symbolEffect(.bounce)</code> na transição
Hints contextuais	<code>.accessibilityHint("Toque duplo para ver detalhes")</code> no link do card
Estado vazio	<code>ContentUnavailableView</code> nativa, totalmente acessível por padrão

Decisões de UI/UX

- **SwiftUI puro, sem UIKit:** usa apenas APIs nativas, garantindo aderência total ao Dynamic Type e ao Modo Escuro sem código extra.
- **LazyVGrid adaptativo:** `GridItem(.adaptive(minimum: 150), spacing: 16)` dá 2 colunas no iPhone retrato, 3 no iPhone landscape ou no iPad, 4+ no iPad Pro horizontal. Tudo sem device check.
- **Imagens via SF Symbols:** cada produto tem um símbolo coerente com sua categoria (`leaf.fill` para madeira, `fork.knife` para comidas, `paintbrush.fill` para arte, etc.). Vantagens: sem peso no bundle, escala perfeitamente no Dynamic Type, segue a paleta do sistema, suporta acessibilidade nativa.
- **Gradientes leves de accent color** nos cards: dão personalidade sem competir com o conteúdo. Funciona bem em claro e escuro.
- **NavigationStack + .searchable embutida na barra:** padrão idiomático moderno do SwiftUI 5 (iOS 17+).
- **Botão de favoritar com `.symbolEffect(.bounce)`** : micro-animação que confirma a ação para usuários videntes, sem interferir no VoiceOver.
- **Tela de detalhes com seções cartonadas:** agrupa metadados (artesão/categoria/preço), descrição e ação principal em "cards" visuais, facilitando varredura visual e leitura linear pelo VoiceOver.

Decisões de implementação

- **ViewModel com `@MainActor`** : evita warnings de concorrência no Swift 6 e garante que mutações de `@Published` rodem na main thread.
- **`produtosFiltrados` como computed property:** derivada de `produtos + termoBusca` , recalcula apenas quando algum deles muda. Evita armazenar estado duplicado.
- **`@ScaledMetric` para altura de imagens:** as imagens crescem proporcionalmente quando o usuário aumenta a fonte do sistema, mantendo a hierarquia visual.
- **Botão favoritar com `.buttonStyle(.plain)`** : necessário para o botão funcionar dentro de um `NavigationLink` sem disparar o link ao clicar no botão.
- **`accessibilityHint` separado do `accessibilityLabel`** : o label descreve o que o elemento é; o hint descreve o que acontece ao tocar. Padrão recomendado pela Apple.

Funcionalidades

Requisitos obrigatórios — atendidos

-  `struct ProdutoArtesanal: Identifiable` com todos os 8 campos exigidos
-  `LazyVGrid` dentro de `ScrollView` com `GridItem(.adaptive(minimum: 150))`
-  Card com imagem, nome, preço formatado e botão favoritar
-  `NavigationLink` para tela de detalhes (exceto ao tocar no botão favoritar)
-  Tela de detalhes completa + botão "Entrar em contato com o Artesão"
-  `@StateObject` + `@Published` para estado reativo dos favoritos
-  `.searchable` filtrando por nome ou categoria

Acessibilidade — atendida

-  `accessibilityLabel` descritivo em todas as imagens
-  Preço acessível (sem leitura literal de "R\$" e "ponto zero zero")
-  Touch targets de 44×44 no botão favoritar
-  Dynamic Type funcional em toda a UI
-  `accessibilitySortPriority` na tela de detalhes (Nome antes da Imagem)

Produtos incluídos

Produto	Categoria	Preço
Escultura de Capivara em Madeira	Madeira	R\$ 85,00
Quibe Frito Tradicional	Comidas	R\$ 12,00
Tela "Calçadas de Curitiba"	Arte	R\$ 250,00
Manta de Tricô Colorida	Vestuário	R\$ 95,00
Relógio de Bolso Antigo	Antiguidades	R\$ 180,00
Cuia de Mate Esculpida	Madeira	R\$ 45,00
Boneca de Pano Maria	Vestuário	R\$ 35,00
Geleia Artesanal de Pinhão	Comidas	R\$ 22,00
Bijuteria com Pedras do Paraná	Acessórios	R\$ 60,00
Vaso de Cerâmica Pintado	Arte	R\$ 75,00
Cinto de Couro Trabalhado	Vestuário	R\$ 110,00
Sabonetes Naturais Trio	Beleza	R\$ 28,00

Dificuldades encontradas

- **Botão dentro de NavLink:** por padrão, o `NavLink` consome o toque em qualquer subview do seu label, fazendo com que tocar no coração também navegasse para a tela de detalhes. A primeira solução com `.buttonStyle(.plain)` em ambos não isolava o tap. Resolvido usando `.buttonStyle(.borderless)` no botão de favoritar — esse estilo permite que o botão filho receba o toque independentemente do container clicável pai, comportamento idiomático do SwiftUI para botões em listas/cards.
- **Estado de favorito não atualizava na tela de detalhes:** passar o produto como cópia por valor (`let produto: ProdutoArtesanal`) congelava o estado no momento da navegação — tocar no coração da toolbar dos detalhes alterava o ViewModel, mas a tela continuava lendo da cópia local. Resolvido recebendo o `viewModel` como `@ObservedObject` e o `produtoId: UUID`, lendo o produto atual via `viewModel.produto(comId:)` sempre que o `body` recompila.
- **Leitura do preço pelo VoiceOver:** ao usar `Text(produto.precoFormatado)`, o VoiceOver lia "R cifrão 45 ponto 00". Resolvido criando a propriedade `precoAcessivel` no model que

retorna texto natural ("Preço: 45 reais") e aplicando

`.accessibilityLabel(produto.precoAcessivel)` no texto.

- **Touch target real:** usar apenas `.frame(minWidth: 44, minHeight: 44)` não basta — a área clicável fica limitada ao ícone. Resolvido adicionando `.contentShape(Rectangle())` para que o tap seja capturado em todo o frame.
- **Ordem de leitura nos detalhes:** o enunciado pede que o Nome seja lido **antes** da Imagem. Usar `.accessibilitySortPriority(10)` no Nome e `(5)` na Imagem inverte a ordem visual sem precisar reorganizar o layout.
- **Dynamic Type em imagens fixas:** imagens com `.frame(height: 130)` ficavam pequenas demais quando o usuário aumentava a fonte. Solução: trocar por `@ScaledMetric(relativeTo: .body) private var alturaImagem: CGFloat = 130`, que escala junto com o tipo do sistema.
- **NumberFormatter recriado por célula:** a primeira versão instanciava `NumberFormatter` dentro do computed property, criando um objeto por acesso. Em um grid com 12+ cards, isso era custoso. Resolvido extraíndo para `private static let precoFormatter` no struct — uma única instância compartilhada.
- **SF Symbol inexistente:** a primeira versão usava `belt` para o cinto de couro, símbolo que não existe no catálogo do iOS — apareceria um placeholder vazio no card. Trocado por `bag.fill`, semanticamente apropriado para acessório.

Roteiro do vídeo

O roteiro detalhado da apresentação está em `ROTEIRO_APRESENTACAO.md` (`ROTEIRO_APRESENTACAO.md`), com divisão de falas entre Lucas Stopinski e Lucas Bruno, timestamps, o que mostrar em cada momento e demos específicos de acessibilidade (VoiceOver + Dynamic Type).

Vídeo de apresentação

(adicionar link do YouTube — não listado, 10–20min)